10-10-2026

INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

Asignatura: Introducción a la Programación

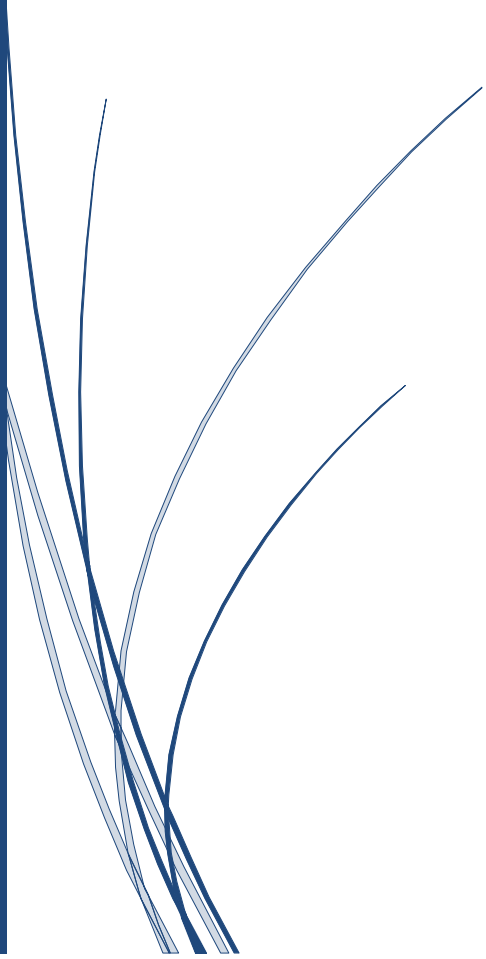
TAREA CAPÍTULO IX: ARREGLOS Y VECTORES

Nombre: Amauri Feliz Valenzuela

Matrícula: 201631629

Grupo o sección: PRG-101-C-2026-03-JG

Profesor: Jimmy Gómez



Índice

Introducción 3

Objetivos 3

9.1 ¿Qué es un arreglo? 3

9.2 Índices y subíndices 4

9.3 Vectores y características 4

Desarrollo del proyecto 5

Explicación del código 6

REPOSITORIO DEL PROYECTO 7

Evidencias de ejecución 8

Casos de prueba 15

Conclusión 16

Fuentes consultadas 17

Introducción

En este trabajo voy a explicar lo que aprendí sobre los arreglos, los índices y los vectores. Durante la investigación comprendí cómo guardar varios datos en una lista, consultar sus posiciones y recorrerlos con ciclos. Para aplicar estos conocimientos, desarrollé un sistema de calificaciones en Python con los nombres y las notas de cinco estudiantes.

En la práctica organicé dos listas relacionadas y un menú para consultar las notas, modificarlas, calcular el promedio y buscar estudiantes. También utilicé condiciones para identificar aprobados y reprobados. A continuación presento los conceptos, los fragmentos de mi código, las capturas de la ejecución y los resultados de las pruebas. Con ellos explico qué hice y cómo funciona el programa.

Objetivos

Objetivo general: comprender cómo almacenar y procesar información mediante listas de Python que representan vectores unidimensionales, y aplicar lo aprendido en un sistema de calificaciones.

Objetivos específicos: relacionar cada nombre con su nota mediante un índice; consultar y modificar elementos; recorrer las listas con for y while; calcular el promedio; utilizar condiciones para contar aprobados y reprobados; buscar estudiantes y comprobar las respuestas del programa ante entradas incorrectas.

9.1 ¿Qué es un arreglo?

Un arreglo reúne varios elementos bajo un mismo nombre y permite consultar sus posiciones. Se utiliza para manejar conjuntos como nombres, temperaturas o calificaciones. En lugar de repetir una operación para muchas variables separadas, se puede recorrer el conjunto. Esto facilita organizar los datos y evita escribir instrucciones similares una y otra vez.

Una variable individual como nota = 85 guarda una sola calificación. Una lista como la siguiente reúne cinco:

```
nota = 85
calificaciones = [85, 90, 78, 95, 88]
print(nota)
print(calificaciones[0])
```

Este ejemplo educativo representa las calificaciones de cinco estudiantes. Contiene cinco elementos y almacena números enteros. Ambas instrucciones de impresión muestran 85, pero la segunda consulta una posición dentro de una colección. Puedo consultar otra posición o recorrer las cinco notas para procesarlas. Estos valores son un ejemplo conceptual; el programa práctico utiliza las notas 85, 67, 92, 74 y 58. Una lista también puede almacenar texto o valores lógicos. En mi programa separé los nombres, que son cadenas de texto, y las notas, que son números enteros.

En otros lenguajes, los arreglos suelen guardar datos del mismo tipo y tener un tamaño fijo. Las listas de Python pueden cambiar de tamaño y contener tipos diferentes. En este ejercicio las utilizo para representar arreglos, aunque no son iguales a los arreglos tipados de otras bibliotecas. Su ventaja es que puedo recorrer las cinco notas sin escribir una instrucción distinta para cada una.

9.2 Índices y subíndices

El índice identifica una posición. En la expresión nombres[2], el número dentro de los corchetes actúa como subíndice y permite acceder al tercer elemento. En este contexto se usan ambos términos para hablar del acceso a una posición.

Python numera las posiciones desde cero por convención. Una forma de comprenderlo es contar cuántas posiciones se avanza desde el comienzo: para el primer elemento no se avanza ninguna. No todos los lenguajes empiezan en cero. Una lista de cuatro elementos tiene longitud 4 y último índice no negativo 3; en general, ese último índice es la longitud menos uno cuando la lista no está vacía.

```
nombres = ["Ana", "Luis", "Carlos", "María"]
print(nombres[0])
print(nombres[2])
nombres[1] = "Pedro"
```

La primera impresión muestra Ana y la segunda Carlos. La última instrucción cambia Luis por Pedro y no imprime nada por sí misma. Después, la lista queda como ["Ana", "Pedro", "Carlos", "María"]. Consultar nombres[4] produce IndexError, porque esa posición no existe. El programa valida el índice antes de acceder a las listas para evitar ese error.

Python también admite índices negativos; por ejemplo, -1 identifica el último elemento. En el menú de mi programa acepto únicamente índices de 0 a 4. Así mantengo una numeración clara y evito confundir una entrada negativa con una consulta válida.

9.3 Vectores y características

En programación introductoria, un vector es un arreglo unidimensional. Unidimensional significa que se necesita un solo índice para ubicar un elemento. No es una tabla de filas y columnas. En mi práctica utilizo listas ordenadas que puedo consultar por posición, recorrer y modificar.

Los elementos conservan un orden y cada posición tiene un índice. Un vector permite almacenar varios valores, consultarlos, modificarlos y recorrerlos en secuencia. En este ejercicio utilizo las listas de Python como vectores unidimensionales; no son arreglos tipados de otras bibliotecas.

El ciclo for visita un elemento en cada iteración:

```
edades = [18, 21, 19, 25, 20]
for edad in edades:
    print(edad)
```

En la primera iteración, edad toma el valor 18 y lo imprime. En la segunda toma 21; en la tercera, 19; en la cuarta, 25; y en la quinta, 20. La salida son esos cinco números, uno por línea. Al terminar los elementos, el ciclo finaliza.

También puedo recorrer el mismo vector con while:

```
edades = [18, 21, 19, 25, 20]
indice = 0
while indice < len(edades):
    print(edades[indice])
    indice += 1
```

Primero inicializo indice en 0. Antes de cada repetición evalúo si es menor que len(edades), que vale 5. Si la condición es verdadera, consulto edades[indice] y lo imprimo. Después aumento el índice en uno. Se visitan las posiciones 0, 1, 2, 3 y 4, y se muestran 18, 21, 19, 25 y 20. Cuando el índice llega a 5, la condición es falsa y el ciclo termina. El incremento evita que se repita indefinidamente la misma posición.

Estos ejemplos me ayudan a comprender las dos formas de recorrer un vector. En mi programa utilicé for para recorrer las listas y while para mantener el menú activo y volver a solicitar una entrada cuando es incorrecta.

Desarrollo del proyecto

Para realizar esta práctica, organicé las calificaciones de cinco estudiantes. Mi propósito fue consultar y actualizar sus notas sin perder la relación entre cada persona y su calificación. Con el programa puedo encontrar información y calcular los resultados sin hacerlo por separado para cada estudiante.

Primero organicé dos listas con cinco elementos: estudiantes para los nombres y calificaciones para las notas. En ambas mantuve el mismo orden. Ana ocupa la posición 0 y su nota, 85, está en esa misma posición de la otra lista. Si cambiara el orden de una sola lista, podría mostrar una nota con el nombre equivocado. Por eso modifico la nota mediante su índice y mantengo esa relación.

Trabajé con los datos de ejemplo del enunciado: Ana, Luis, Carlos, Maria y Pedro, con notas de 85, 67, 92, 74 y 58. Para comparar los resultados, partí de esos valores y probé cambios en algunas calificaciones.

Para el ejercicio establecí una escala de 0 a 100 con notas enteras y un mínimo de 70 para aprobar. Esta es una regla de mi programa, no una norma oficial del ITLA. Guardé los datos en las listas durante la ejecución: al salir se pierden los cambios y, cuando inicio otra sesión, aparecen nuevamente las notas originales.

Herramientas y funcionamiento

Después de organizar las listas, desarrollé un menú dentro de un ciclo while para poder elegir una operación y continuar hasta seleccionar la salida. Separé las tareas en funciones sencillas: mostrar las notas, calcular el promedio, buscar un estudiante y leer un número válido. En main.py puedo seguir cada función y comprender para qué la utilicé.

Para realizar la práctica utilicé Python y Visual Studio Code. Abrí el archivo en el editor y ejecuté py main.py desde la terminal integrada de Windows. Con Git organicé los archivos del proyecto y mediante GitHub compartí el código para que el profesor pueda consultarlo.

Organicé el menú en diez operaciones. Con la opción 1 muestro los estudiantes y sus notas; con la 2 consulto un índice; y con la 3 obtengo el promedio. Las opciones 4 y 5 muestran la nota más alta y la más baja. Con la 6 y la 7 consulto y cuento aprobados y reprobados. En la 8 modifico una nota; en la 9 busco un nombre; y en la 10 obtengo el resumen actualizado. Para terminar utilizo la opción 0.

Agregué leer_entero para solicitar un número entero y comprobar sus límites. Si escribo letras o un decimal, la función captura ValueError y vuelve a preguntar. Si introduzco un valor fuera del rango, recibo un mensaje con los límites permitidos. Así puedo corregirlo antes de consultar una posición de la lista. También incluí un mensaje de cierre si interrumpo la entrada o utilizo el teclado para detener el programa.

Explicación del código

1. Creación de los vectores

```
estudiantes = ["Ana", "Luis", "Carlos", "Maria", "Pedro"]
calificaciones = [85, 67, 92, 74, 58]
```

Guardo cinco nombres en estudiantes y cinco notas en calificaciones. Uso el mismo índice para relacionar ambas listas. Al mostrarlas obtengo Ana 85, Luis 67, Carlos 92, Maria 74 y Pedro 58. Así puedo almacenar varios datos sin crear una variable para cada estudiante.

2. Acceso mediante índices

```
indice = leer_entero("Índice del estudiante (0 a 4): ", 0, 4)
print(f"{estudiantes[indice]}: {calificaciones[indice]} puntos")
```

Primero introduzco una posición válida y después consulto ese índice en las dos listas. Si escribo 2, obtengo Carlos: 92 puntos. Así puedo encontrar directamente el nombre y la nota que corresponden a una posición.

3. Recorrido mediante ciclos

```
for indice in range(len(estudiantes)):
    print(f"{indice}. {estudiantes[indice]}: {calificaciones[indice]} puntos")
```

Con len obtengo la cantidad de estudiantes, que es cinco. range(5) genera los índices 0, 1, 2, 3 y 4. En cada repetición utilizo el índice para mostrar un nombre y su nota. Al terminar, aparecen los cinco estudiantes del listado.

4. Condiciones y conteo

```
aprobo = calificaciones[indice] >= MINIMO_APROBACION
if aprobo == aprobados:
    print(f"{estudiantes[indice]}: {calificaciones[indice]} puntos")
    cantidad = cantidad + 1
```

Utilicé una comparación para saber si cada nota alcanza los 70 puntos. Con el parámetro aprobados elijo el grupo: True para aprobados y False para reprobados. Cuando encuentro una nota del grupo elegido, muestro el nombre y aumento cantidad en uno. Con los valores iniciales obtengo tres aprobados y dos reprobados. También utilicé if, elif y else para elegir las operaciones del menú.

5. Cálculo del promedio

```
total = 0
for nota in calificaciones:
    total = total + nota
return total / len(calificaciones)
```

Para calcular el promedio, empecé con total en cero y utilicé un ciclo for para sumar las calificaciones. La suma inicial es 85 + 67 + 92 + 74 + 58 = 376; 376 / 5 = 75.2. El formato de impresión muestra 75.20. Con este cálculo relaciono el recorrido de la lista con la suma y la división. La lista debe tener elementos para poder dividir entre su longitud; en este programa siempre hay cinco notas.

6. Modificación

```
anterior = calificaciones[indice]
calificaciones[indice] = nueva
print(f"{estudiantes[indice]}: antes {anterior}, ahora {nueva} puntos")
```

Primero introduzco un índice y una nota dentro de los límites permitidos. Al elegir el índice 1 y escribir 80, aparece el mensaje Luis: antes 67, ahora 80 puntos. La lista queda [85, 80, 92, 74, 58] y el promedio pasa a 77.80. Cambio solamente la nota de esa posición, por lo que el nombre de Luis sigue relacionado con su calificación.

7. Búsqueda

```
for indice in range(len(estudiantes)):
    if estudiantes[indice].lower() == nombre.strip().lower():
        return indice
return -1
```

Para buscar un estudiante, comparo los nombres uno por uno. lower permite buscar Luis escribiendo luis, y strip elimina espacios al comienzo y al final de la entrada. Si busco a Luis, obtengo su índice, que es 1. Si no encuentro el nombre, la función devuelve -1 y muestro Estudiante no registrado. Conservo Maria sin tilde en los datos iniciales; la búsqueda no elimina las tildes, aunque sí permite escribir el nombre con distintas mayúsculas. Así utilizo un ciclo y una condición para encontrar el nombre que busco.

8. Validación de índices

```
def leer_entero(mensaje, minimo, maximo):
    while True:
        try:
            valor = int(input(mensaje))
        except ValueError:
            print("Entrada incorrecta: escriba un número entero.")
            continue
        if minimo <= valor <= maximo:
            return valor
        print(f"Valor fuera de rango: debe estar entre {minimo} y {maximo}.")
```

Con esta función solicito un entero y acepto el dato únicamente cuando está dentro de los límites indicados. Para consultar una nota utilicé el rango de 0 a 4. Si escribo letras, recibo un mensaje para introducir un entero; si escribo 9, obtengo Valor fuera de rango y puedo intentarlo otra vez. Utilicé while para repetir la pregunta y evitar que un índice inexistente produzca IndexError.

REPOSITORIO DEL PROYECTO

Nombre del proyecto: Sistema de Calificaciones en Python.

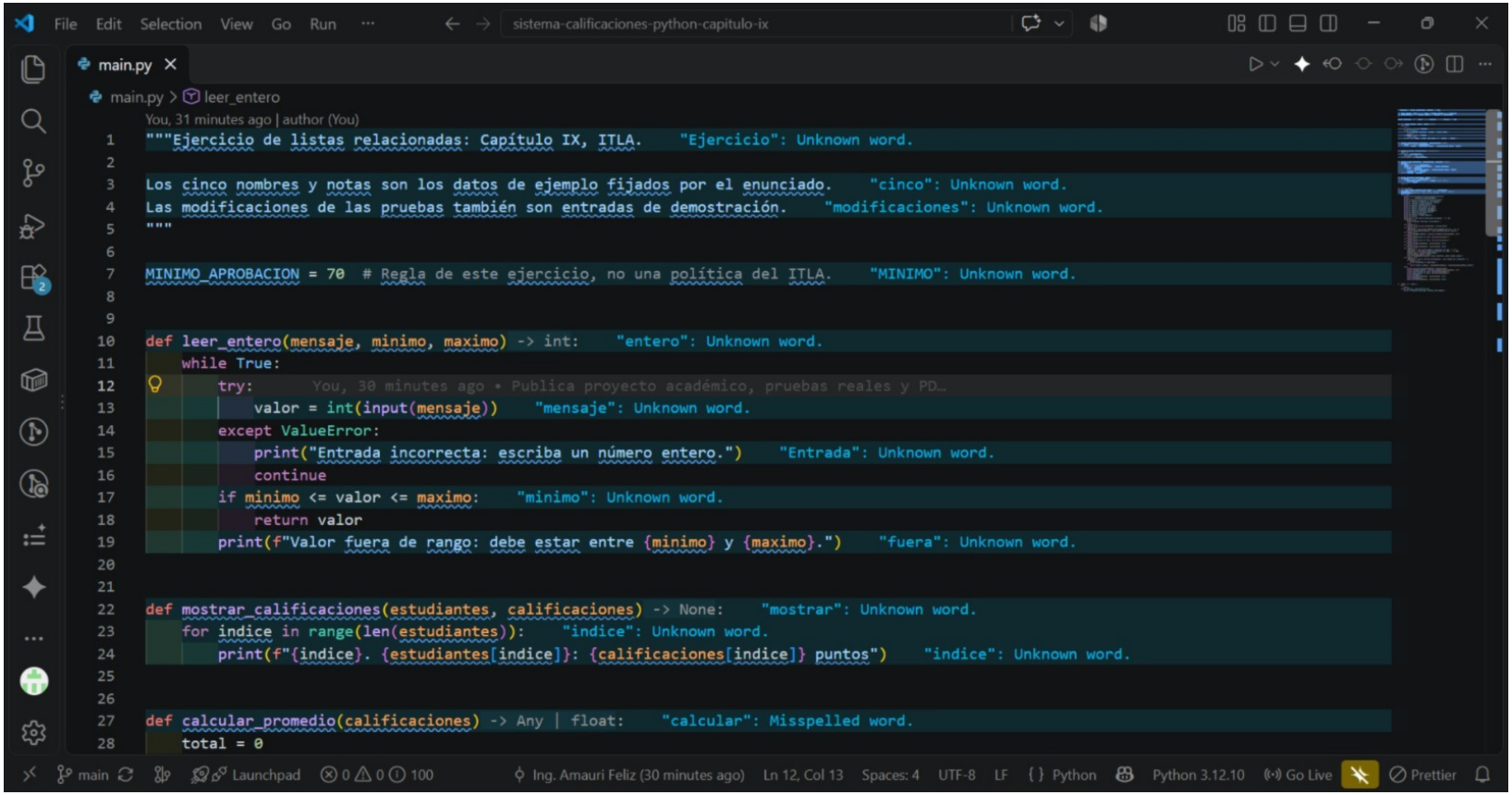
Repositorio GitHub: https://github.com/mrcarlos1990/sistema-calificaciones-python-capitulo-ix

Publiqué el proyecto en un repositorio público de GitHub para que el profesor pueda consultar mi código sin solicitar permisos. Incluí main.py y un README con la descripción, el objetivo, las funciones y las instrucciones para ejecutar el programa. Para usarlo en Windows, puedo abrir la carpeta en Visual Studio Code y escribir py main.py en la terminal integrada.

Evidencias de ejecución

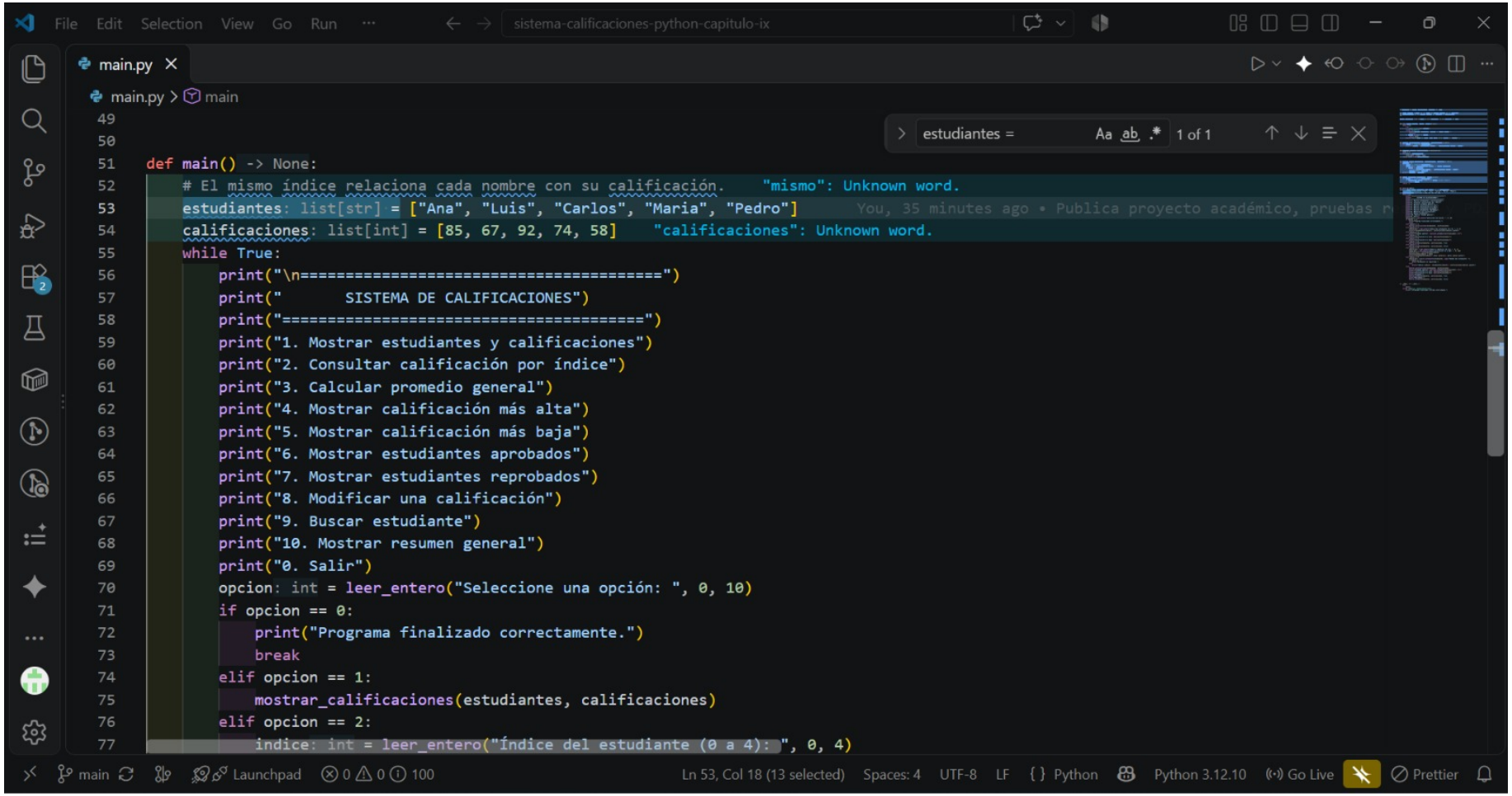
Durante la ejecución en Visual Studio Code tomé estas siete capturas para mostrar cómo trabajé con el programa. En ellas presento el código, las listas, el menú y los resultados. También muestro cómo cambié una nota, corregí un índice incorrecto y finalicé la ejecución.

Figura 1. Código fuente desarrollado en Python.



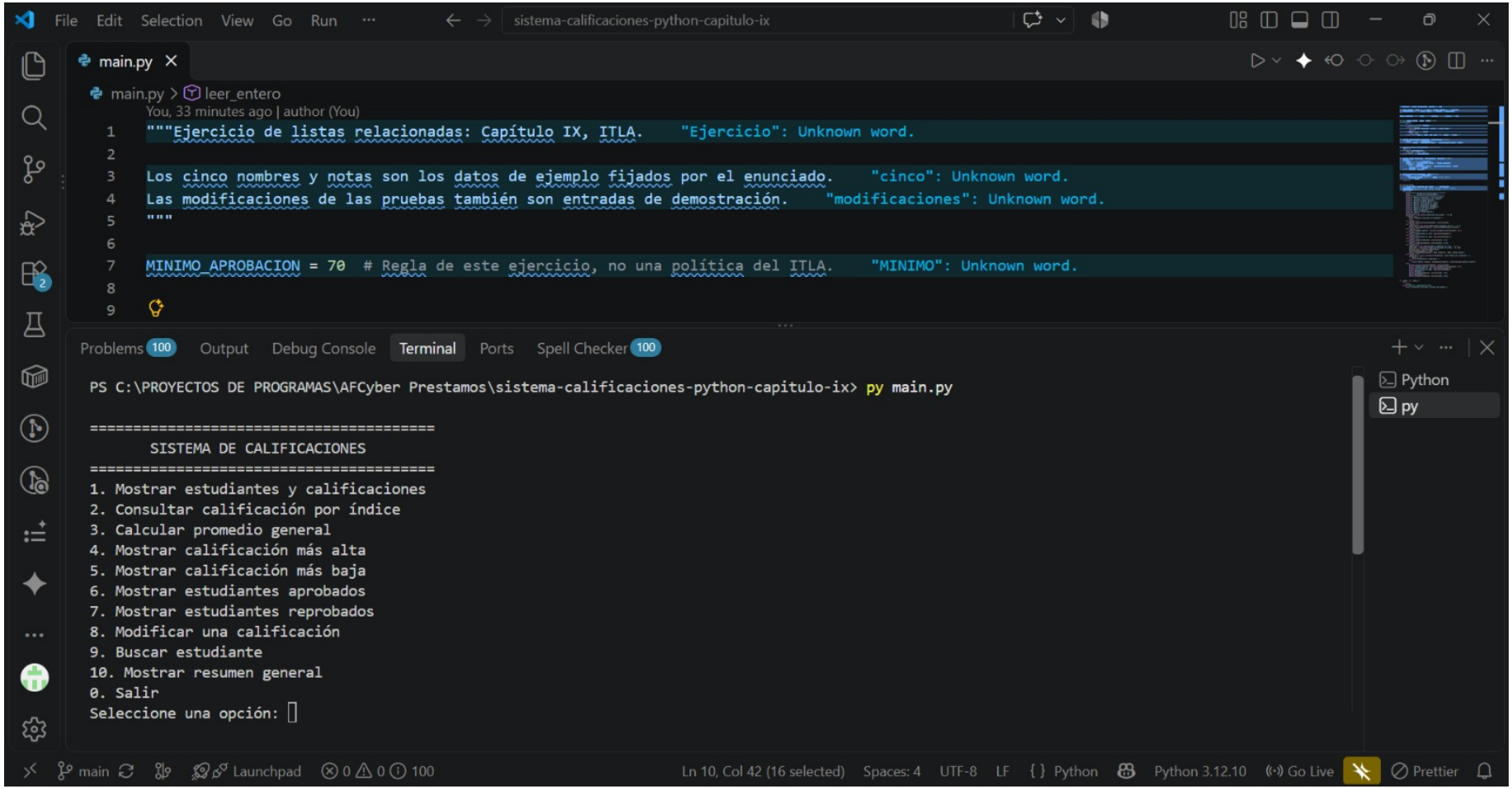
Abrí main.py en Visual Studio Code. En esta parte de mi código utilicé leer_entero para aceptar entradas válidas y for para mostrar las calificaciones.

Figura 2. Creación de los vectores.



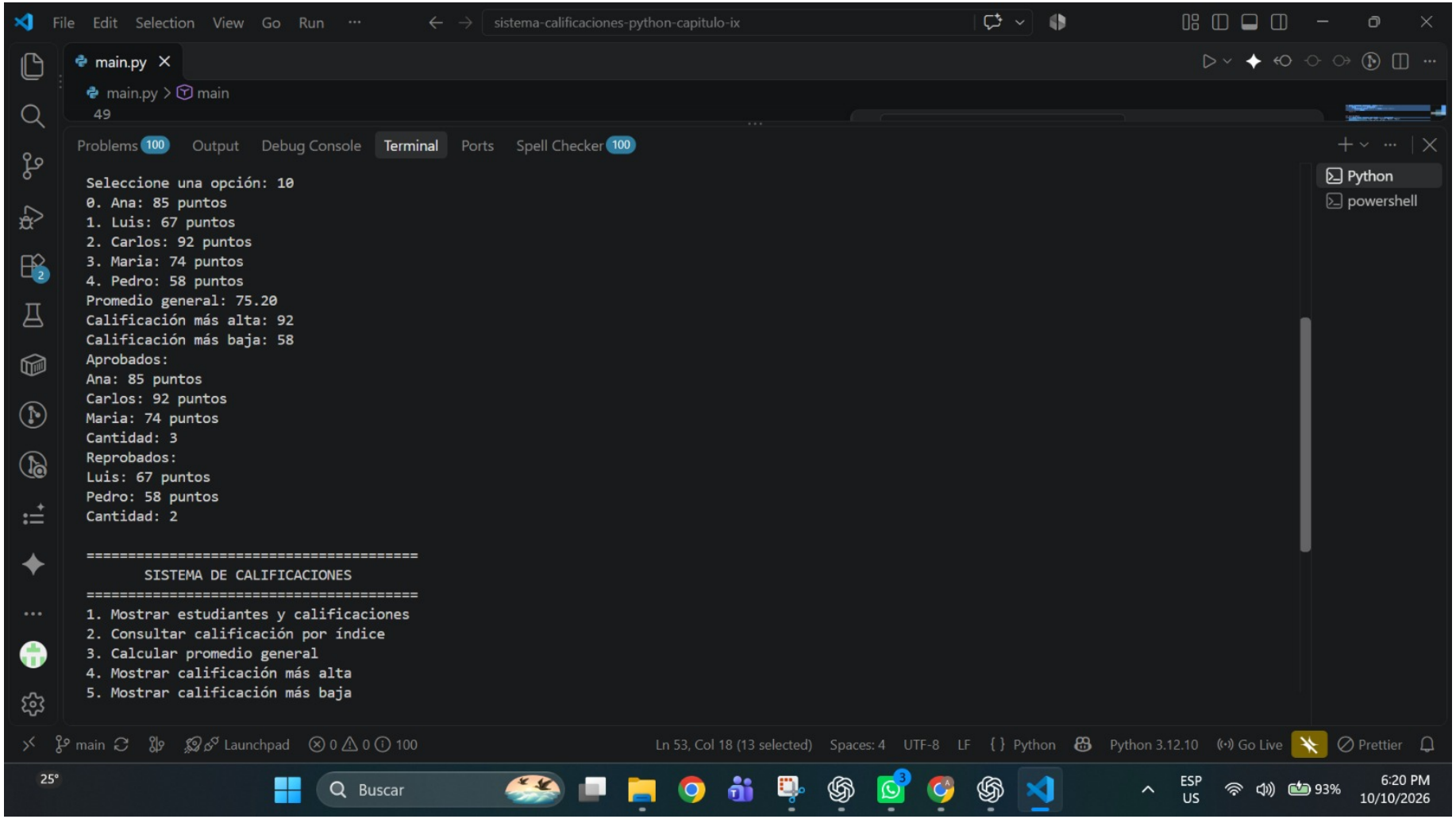
Organicé los cinco nombres y las cinco notas en dos listas. Utilicé el mismo índice para relacionarlas; por ejemplo, Ana ocupa la posición 0 y tiene 85 puntos.

Figura 3. Ejecución inicial del programa.



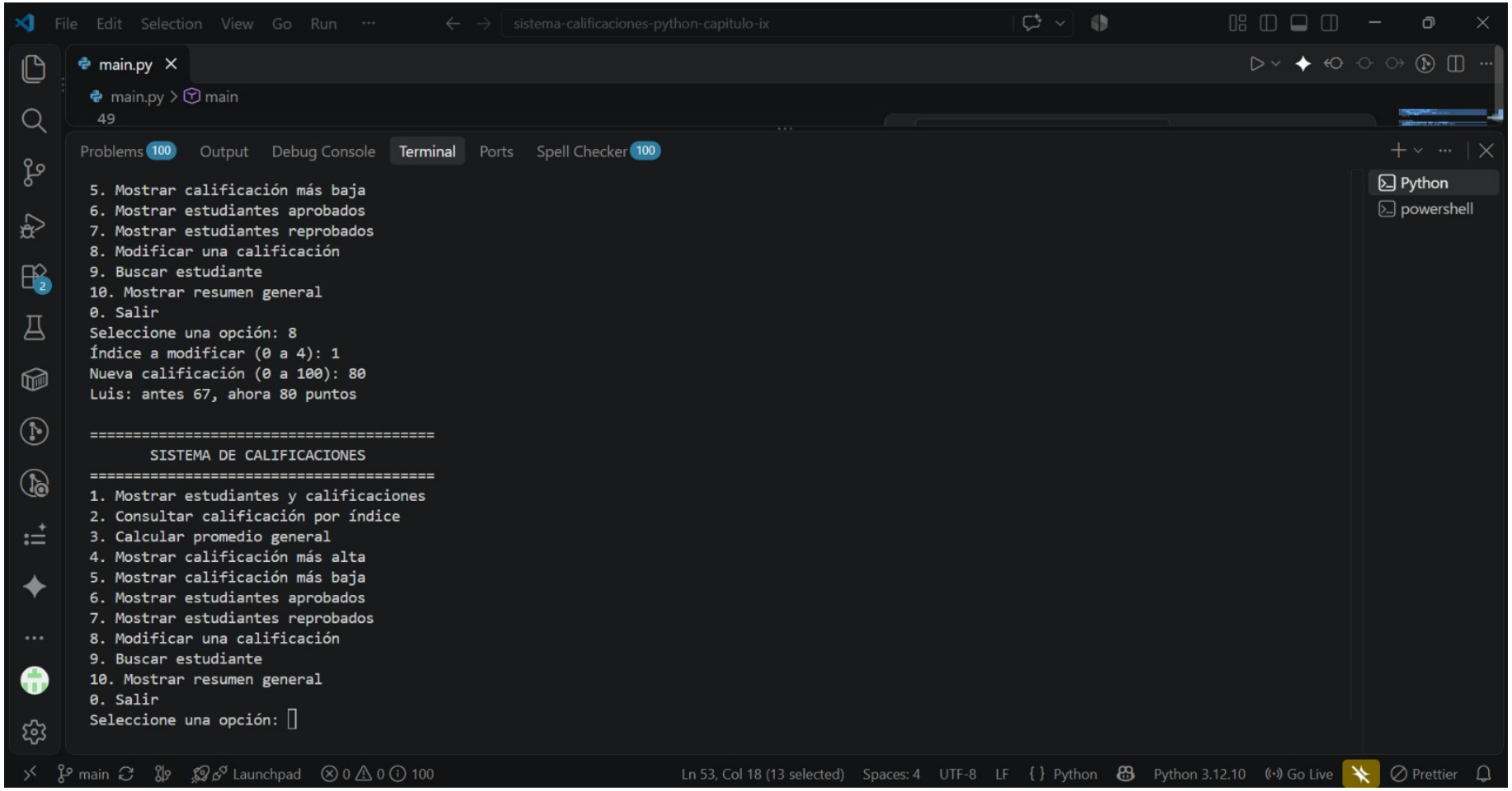
Ejecuté el programa desde la terminal integrada de Visual Studio Code. Observé el menú con sus diez operaciones y la opción 0 para salir.

Figura 4. Resultados generales de las calificaciones.



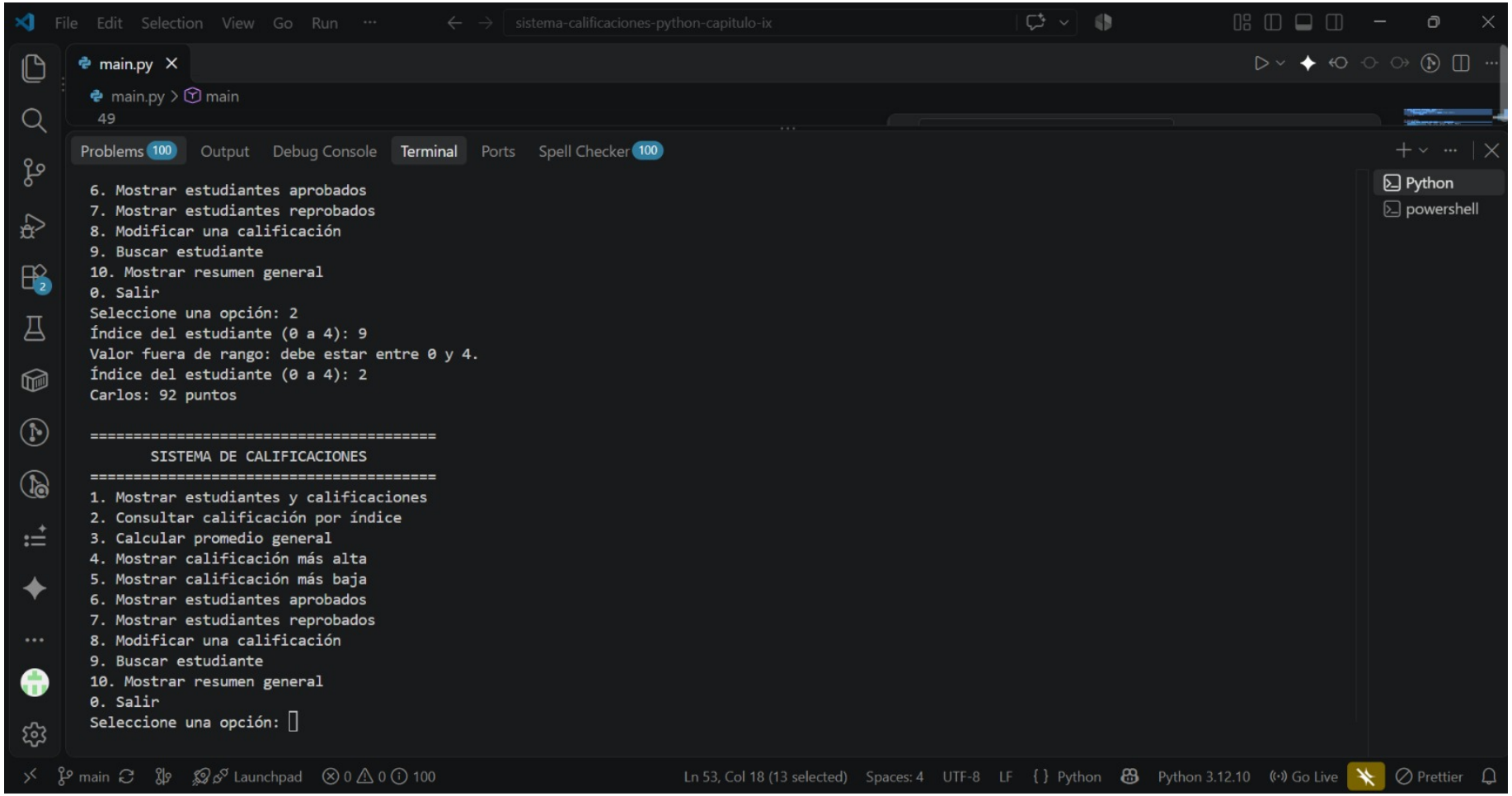
Seleccioné el resumen con las calificaciones originales. Obtuve un promedio de 75.20, una nota máxima de 92 y una mínima de 58, con tres aprobados y dos reprobados.

Figura 5. Modificación de una calificación.



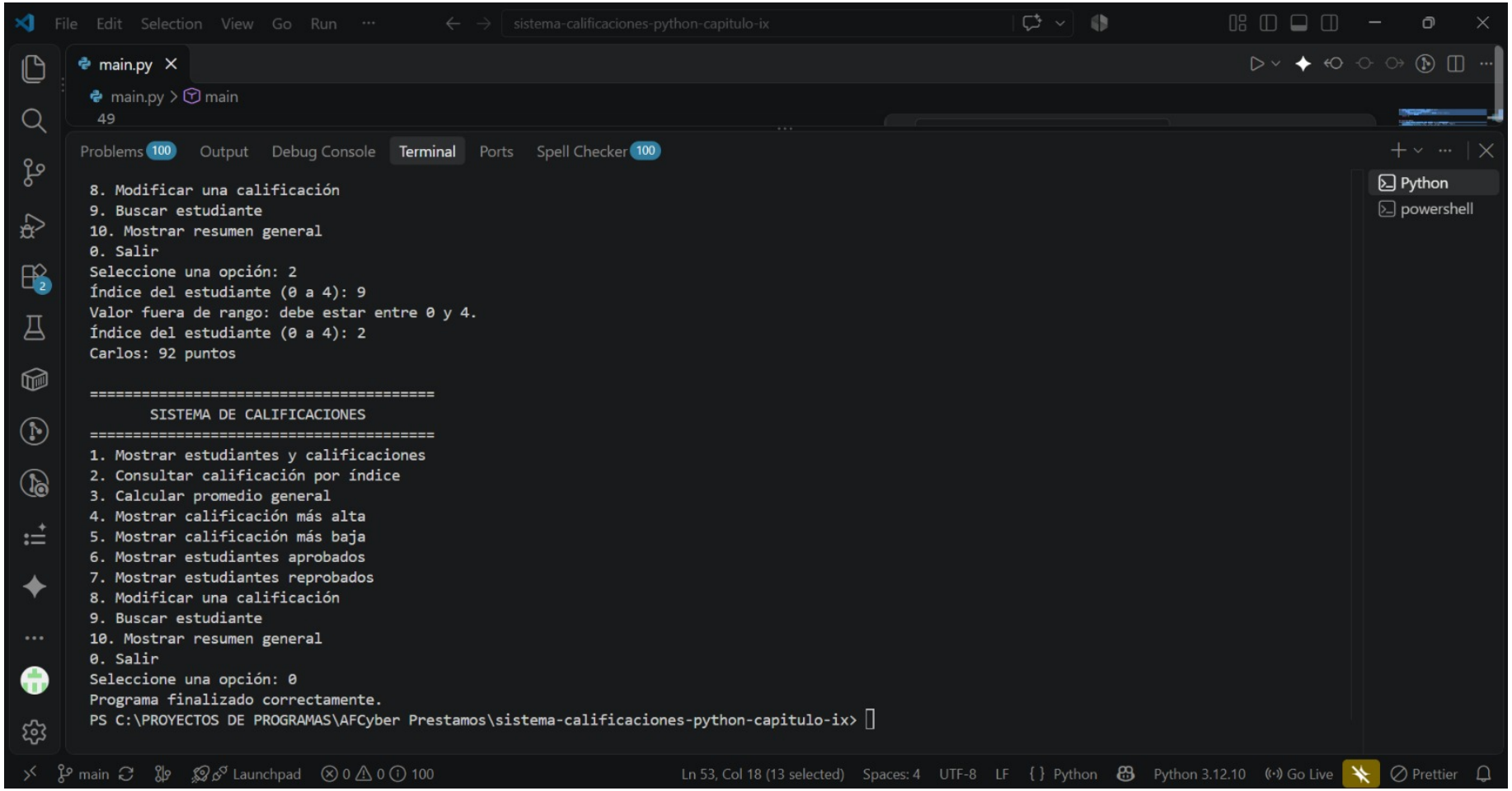
Elegí la opción 8, el índice 1 y la nota 80. Observé el mensaje que muestra el cambio de Luis de 67 a 80 puntos y pude continuar utilizando el menú.

Figura 6. Validación de índices incorrectos.



Escribí el índice 9 y recibí el mensaje de valor fuera de rango. Lo corregí a 2 y consulté la nota de Carlos, que es 92. Pude continuar sin cerrar el programa.

Figura 7. Finalización correcta del programa.



Elegí la opción 0 para salir. Observé Programa finalizado correctamente y regresé a la terminal; así comprobé que el menú terminó.

Casos de prueba

Para comprobar mi programa utilicé once casos de prueba y comparé los resultados esperados con los obtenidos. Revisé el listado, el promedio, la consulta por índice, la modificación de notas, los aprobados y reprobados, la búsqueda y la salida. También observé cómo responde ante entradas incorrectas.

En cada caso partí de las calificaciones originales. Al cambiar la nota de Luis de 67 a 80, comprobé que el listado muestra 80 y que el promedio pasa de 75.20 a 77.80. Con las notas iniciales obtuve tres aprobados y dos reprobados. Los casos con notas de 0, 100 y 70 me permitieron revisar los límites y el mínimo de aprobación. Al introducir un índice inexistente, pude corregirlo y continuar. Los once casos de la tabla terminaron correctamente.

En la columna Datos utilizados, los valores separados por punto y coma indican las entradas en el orden en que se escriben. Por ejemplo, 2; 2; 0 significa elegir la consulta por índice, escribir el índice 2 y salir con la opción 0.

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
1. Visualización inicial	1; 0	0. Ana: 85; 1. Luis: 67; 2. Carlos: 92; 3. Maria: 74; 4. Pedro: 58	0. Ana: 85; 1. Luis: 67; 2. Carlos: 92; 3. Maria: 74; 4. Pedro: 58	APROBADA
2. Promedio	3; 0	Promedio general: 75.20	Promedio general: 75.20	APROBADA
3. Consulta por índice	2; 2; 0	Carlos: 92 puntos	Carlos: 92 puntos	APROBADA
4. Modificación	8; 1; 80; 1; 3; 0	Luis: antes 67, ahora 80; 1. Luis: 80; Promedio general: 77.80	Luis: antes 67, ahora 80; 1. Luis: 80; Promedio general: 77.80	APROBADA
5. Aprobados y reprobados	6; 7; 0	Ana: 85; Carlos: 92; Maria: 74; Cantidad: 3; Luis: 67; Pedro: 58; Cantidad: 2	Ana: 85; Carlos: 92; Maria: 74; Cantidad: 3; Luis: 67; Pedro: 58; Cantidad: 2	APROBADA
6. Validación y continuidad	letras; 99; 2; 9; -1; 2; 8; 0; -5; 101; abc; 90; 1; 0	escriba un número entero; entre 0 y 10; entre 0 y 4; entre 0 y 100; Ana: antes 85, ahora 90; 0. Ana: 90	escriba un número entero; entre 0 y 10; entre 0 y 4; entre 0 y 100; Ana: antes 85, ahora 90; 0. Ana: 90	APROBADA
7. Búsqueda	9; lUiS ; 9; Sofia; 0	Índice 1. Luis: 67; Estudiante no registrado.	Índice 1. Luis: 67; Estudiante no registrado.	APROBADA
8. Extremos y resumen	10; 0	Promedio general: 75.20; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 3; Cantidad: 2	Promedio general: 75.20; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 3; Cantidad: 2	APROBADA
9. Límites y aprobación en 70	8; 0; 0; 8; 1; 100; 8; 4; 70; 10; 0	Ana: antes 85, ahora 0; Luis: antes 67, ahora 100; Pedro: antes 58, ahora 70; Promedio general: 67.20; Cantidad: 4; Cantidad: 1	Ana: antes 85, ahora 0; Luis: antes 67, ahora 100; Pedro: antes 58, ahora 70; Promedio general: 67.20; Cantidad: 4; Cantidad: 1	APROBADA

Casos de prueba (continuación)

Prueba	Datos utilizados	Resultado esperado	Resultado obtenido	Estado
10. Resumen después de modificar a Luis	8; 1; 80; 10; 0	Luis: antes 67, ahora 80; 1. Luis: 80 puntos; Promedio general: 77.80; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 4; Cantidad: 1	Luis: antes 67, ahora 80; 1. Luis: 80 puntos; Promedio general: 77.80; Calificación más alta: 92; Calificación más baja: 58; Cantidad: 4; Cantidad: 1	APROBADA
11. Salida con opción 0	0	Programa finalizado correctamente.	Programa finalizado correctamente.	APROBADA

Conclusión

¿Qué aprendí sobre los arreglos?

Aprendí que puedo reunir varios datos bajo un mismo nombre y trabajar con ellos por sus posiciones. En este ejercicio utilicé listas de Python para representar vectores: una guarda los nombres y otra las calificaciones.

¿Por qué son importantes los índices?

Los índices me permiten localizar, consultar y modificar un elemento específico. En las dos listas del programa, el mismo índice relaciona a un estudiante con su nota. Comprendí que la primera posición es 0 y que cinco elementos tienen como último índice 4.

¿Qué ventajas ofrecen los vectores?

Los vectores me ayudan a organizar varios valores y a procesarlos sin crear una variable independiente para cada estudiante. Puedo recorrer las cinco calificaciones con un ciclo y aplicar la misma operación a cada una.

¿Cómo se relacionan los ciclos con los arreglos?

Con for recorrí las listas para mostrar las notas, sumarlas y buscar un nombre. Utilicé while para mantener el menú disponible y volver a solicitar un dato incorrecto. Con las condiciones elegí cada operación e identifiqué a quienes alcanzan el mínimo de 70 puntos que establecí para el ejercicio.

¿Qué dificultad encontré desarrollando el programa?

Una dificultad que tuve fue comprender cómo funcionan los índices en Python, porque las posiciones comienzan desde cero y no desde uno. También tuve que prestar atención a lo que sucedía cuando se introducía un índice que no existía en la lista.

¿Cómo solucioné esa dificultad?

Para resolverlo, revisé el código, realicé diferentes pruebas y utilicé una validación que detecta valores fuera del rango establecido. Así comprendí mejor cómo acceder, consultar y modificar los elementos de un arreglo sin que el programa se cierre por un índice incorrecto.

¿En qué aplicación real podría utilizar arreglos?

Podría utilizarlos para organizar calificaciones, cantidades de productos en un inventario o registros de ventas. En el sistema desarrollado apliqué listas, índices, asignaciones, ciclos, condiciones, operaciones matemáticas, búsqueda y validación. El ejemplo funciona durante una sesión de consola y me permite explicar cómo se relacionan esos conocimientos para resolver un problema sencillo.

Fuentes consultadas

Python Software Foundation. Una introducción informal a Python, sección Listas. Documentación oficial. https://docs.python.org/es/3/tutorial/introduction.html. Consulta: 10 de octubre de 2026. Recurso utilizado para verificar listas, acceso por posición y modificación.

Python Software Foundation. Estructuras de datos. Tutorial oficial de Python. https://docs.python.org/es/3/tutorial/datastructures.html. Consulta: 10 de octubre de 2026. Referencia para las operaciones sobre listas.

Python Software Foundation. Más herramientas para control de flujo. Tutorial oficial de Python. https://docs.python.org/es/3/tutorial/controlflow.html. Consulta: 10 de octubre de 2026. Referencia para for, condiciones y control de ciclos.

Estas fuentes me sirven de apoyo para comprender las listas, los índices y los ciclos de Python. Explico los conceptos con mis propias palabras y los relaciono con el programa de calificaciones.